ML Systems Lab

A hands-on performance study of LLM inference and GPU systems, built on free Google Colab and GitHub.

Repository: https://github.com/shay2222/ml-systems-lab · October 2026

1. Purpose

ML Systems Lab answers one question for each part of modern model serving: where does time go, and what changes it? Each of five experiments asks a specific question, measures the answer on real hardware, and records the result with its limitations. The project is deliberately small so anyone with a free Colab account can reproduce every number.

The goal is not to beat production systems. It is to show that I can measure, explain and improve performance at the levels an ML systems team works on: the inference loop, batching and scheduling, GPU kernels, profiling, and distributed training.

2. Results at a glance

#	Experiment	Key measured result
01	KV cache	1.51x faster at 700 tokens (29.5 vs 19.5 tok/s); about equal at 200 tokens. The cache matters more as output grows.
02	Batching	Total throughput 22.2 to 439 tok/s at batch 1 to 16 (about 20x). Each user waits only about 6% longer at batch 16.
03	Triton RMSNorm kernel	9 to 13x faster than unfused PyTorch; 1.1 to 4.2x faster than torch.compile. Correct to max difference 0.0039 (fp16).
04	Profiling one forward pass	Matrix multiplies (aten::mm) 54.9% of GPU time; scaled-dot-product attention 15.5%; copy_ and mul called hundreds of times per pass.
05	Data-parallel training	2 CPU processes gave 1.27x throughput (about 63% efficiency). Gradient all-reduce was 18% of step time.

Hardware: Tesla T4 GPU on free Colab for experiments 01 to 04; CPU runtime for 05. Model: Qwen2.5-0.5B-Instruct (fp16). Most rows are single runs; results vary between runs, so small differences are treated as noise.

3. What each experiment shows

01 KV cache. Without a cache, each new token recomputes attention over the whole history. Measured speedup grows with output length (0.66x to 1.51x from 200 to 700 tokens; the 200-token result is within noise). The cache trades memory for compute, and its value depends on sequence length.

02 Batching. A single request leaves the GPU mostly idle, so total tokens per second scale nearly linearly with batch size. Per-user latency rose only from 7.57 s at batch 1 to 8.05 s at batch 16. This is the core trade-off in serving: a large throughput gain for a small latency cost.

03 Triton kernel. PyTorch runs RMSNorm as several separate GPU operations, each reading and writing memory. The Triton kernel fuses them into one pass per row. Against plain PyTorch the gain is 9 to 13x. Against torch.compile, which also fuses, the gain shrinks to 1.1 to 4.2x and nearly disappears at 16384 x 896. Reporting the compiled comparison makes the result credible: most of the eager-mode gap is fusion.

04 Profiling. Reading the CUDA column rather than the CPU column changed the picture: attention is about 15% of GPU time. The profile points to the next targets: unfused attention (the math path, not a fused FlashAttention-style kernel) and the high call counts of copy_ and mul.

05 Data-parallel training. Two processes synchronized gradients with all-reduce. Throughput rose 27% rather than doubling, because all-reduce cost about 6.9 ms per step and the processes shared two CPU

cores. The result is reported as measured, with the CPU-only limitation stated.

4. Technical skills and tools

Area	Tools and techniques used	Where it appears (file)
LLM inference	Hugging Face transformers, KV cache (use_cache), greedy decoding, chat templates, padded batching	01-kv-cache/notebook.ipynb; 02-batching/notebook.ipynb
Serving and scheduling	Static batching, throughput vs latency measurement, per-user timing, batch-size sweeps	02-batching/notebook.ipynb; 02-batching/README.md
GPU kernels	Triton (@triton.jit), program IDs, masked loads and stores, fp16 to fp32 accumulation, block sizing	03-triton-kernel/notebook.ipynb
Compiler baselines	torch.compile fusion; comparing eager vs compiled vs hand-written kernels	03-triton-kernel/notebook.ipynb
CUDA concepts	torch.cuda.synchronize for correct timing, warm-up runs, memory-bound vs compute-bound, GEMM kernels	02-batching/notebook.ipynb; 03-triton-kernel/notebook.ipynb; 04-profiling/notebook.ipynb
Profiling	torch.profiler with CPU and CUDA activity, Self CPU vs Self CUDA, avoiding double-counted wrapper ops	04-profiling/notebook.ipynb; 04-profiling/README.md
Distributed training	torch.distributed, DistributedDataParallel, gloo backend, torchrun, all-reduce, barriers	05-distributed/ddp_test.py; 05-distributed/notebook.ipynb
Numerical correctness	Reference-implementation checks, max absolute difference, allclose tolerances	03-triton-kernel/notebook.ipynb
Engineering practice	Benchmark harnesses, reproducible notebooks, git versioning, stated limitations in every README	README.md (repo root) and every folder README

5. How this shows me as a performance engineer

- **I measure before I claim.** Every result has a number, a method (warm-up, averaged runs, synchronization), and a stated limitation.
- **I find the bottleneck, then test a fix.** The profile identified matrix multiplies and copies; the kernel and batching work targeted memory traffic and idle GPU time.
- **I use fair baselines.** I compared against torch.compile, not only plain PyTorch, and reported the smaller margin.
- **I verify correctness first.** A faster kernel only counts if its output matches the reference.
- **I explain trade-offs.** KV cache memory vs compute, throughput vs per-user latency, communication cost vs parallel speedup.
- **I connect layers.** The work moves from Python-level inference, to batching and scheduling, to GPU kernels, to distributed execution.

6. Where this applies

- **LLM serving teams:** batching, KV cache sizing and latency-throughput tuning directly change cost per token.
- **GPU and kernel optimization:** fusing memory-bound operations such as normalization, activation and attention, and comparing against compiler output.
- **Profiling and cost reduction:** finding where GPU time goes so effort goes to the largest share first.
- **Training infrastructure:** data-parallel scaling, communication overhead and efficiency measurement.
- **Any ML product team:** the same discipline applies to latency budgets, capacity planning and regression testing after model or prompt changes.

7. What this project can become

The five folders are a foundation. Each one has a clear next step that would turn it from a study into a tool or a stronger result:

- **A decode-phase profiler (04):** profile one generated token at a time, where the breakdown differs from the prefill pass measured here, and compare it across context lengths.
- **A fused attention kernel (03 to 04):** replace the unfused math attention path with a Triton kernel and measure it against PyTorch's fused scaled-dot-product attention.
- **A continuous-batching scheduler (02):** let new requests join a running batch when a sequence finishes, and measure p95 latency under a realistic arrival rate.
- **A quantization study:** compare fp16 with int8 or int4 weights on speed, memory and accuracy, the most common cost lever in production inference.
- **Multi-GPU scaling (05):** rerun the distributed test on two or more GPUs to measure real scaling efficiency, and add tensor or pipeline parallelism.
- **A reproducible benchmark suite:** one command that reruns every experiment and writes a results table, with repeated runs and confidence intervals.

8. Future value

Beyond the portfolio, the project is a working template. The same pattern (state a question, measure with a fair baseline, record limitations, publish a reproducible notebook) can be applied to any model-serving or kernel problem a team brings. It also gives a shared vocabulary for discussions about latency, throughput, memory bandwidth and cost, which are the terms most ML infrastructure decisions are made in.

9. Limitations and next steps

- Single model, single GPU type, free-tier Colab, and single runs in several tables. Read the results as directional.
- Experiment 05 ran on CPU only; true multi-GPU scaling was not measured.
- The profile covers one prefill-style forward pass; the decode loop would show a different breakdown.
- Next steps are listed in section 7, in priority order: decode profiling, fused attention, and continuous batching.